CS228 Logic for Computer Science
Exercise Problem Set 9

Conventions.

- Programs, states, formulas and proof rules are as in Lectures 19–21.
- In Hoare triples, for us, the pre- and postconditions are first-order formulas over the integers.
- We write $*$ for multiplication in programs and $\cdot$ in formulas. In formulas we may use $x!$, with $0! = 1$ and $n! = n \cdot (n-1)!$ for $n \geq 1$.
- $\vdash_{par}$ denotes derivability in the proof calculus for partial correctness.
- Write every derivation as a *proof outline* as in Lecture 20: each formula on its own line, with the name of the rule used next to it.
- An Implied step needs an implication that is valid over the integers; you may justify it by ordinary arithmetic reasoning.

Part 1

1. Prove each of the following using only the rules Assignment, Sequential Composition and Implied.
   (a) $\vdash_{par} \{x > 0\}\ y := x + 1\ \{y > 1\}$
   (b) $\vdash_{par} \{\top\}\ y := x;\ y := x + x + y\ \{y = 3 \cdot x\}$
   (c) $\vdash_{par} \{x > 1\}\ a := 1;\ y := x;\ y := y - a\ \{y > 0 \wedge x > y\}$

2. (a) Show that
   $$\vdash_{par} \{x = x_0 \wedge y = y_0\}\ x := x + y;\ y := x - y;\ x := x - y\ \{x = y_0 \wedge y = x_0\},$$
   i.e., these three assignments swap x and y without a temporary variable. Which of x, y, x_0, y_0 are logical variables?
   (b) Write a program **Plus2** such that $\vdash_{par} \{\top\}$ **Plus2** $\{y = x + 2\}$, and prove that this triple holds. **Plus2** must terminate from every state. (Otherwise there is a cheap answer: a program that never terminates satisfies every partial-correctness triple.)
   (c) Write a program **Prog-inc** such that $\vdash_{par} \{\top\}$ **Prog-inc** $\{z > x + y + 4\}$, and prove that this triple holds. Again, **Prog-inc** must terminate from every state.

3. Recall the rule If-then-Else from Lecture 20: from $\{\varphi \wedge B\}\ C_1\ \{\psi\}$ and $\{\varphi \wedge \neg B\}\ C_2\ \{\psi\}$ conclude $\{\varphi\}$ **if** B $\{C_1\}$ **else** $\{C_2\}$ $\{\psi\}$.
   (a) Prove $\vdash_{par} \{\top\}$ **if** $(x \leq y)$ $\{z := y\}$ **else** $\{z := x\}$ $\{z \geq x \wedge z \geq y\}$ (the triple from Problem 5(b) of Problem Set 8).
   (b) Let **Succ** be the program $a := x + 1;$ **if** $(a - 1 = 0)$ $\{y := 1\}$ **else** $\{y := a\}$. Prove $\vdash_{par}$ $\{\top\}$ **Succ** $\{y = x + 1\}$ *without* using the variant of If-then-Else from Lecture 21, i.e., using only the rule stated above. What midcondition do you need after $a := x+1$, and why is $\top$ not enough?

(c) The *variant* of the rule says: from $\{\varphi_1\}\ C_1\ \{\psi\}$ and $\{\varphi_2\}\ C_2\ \{\psi\}$ conclude $\{(B \to \varphi_1) \wedge (\neg B \to \varphi_2)\}$ **if** B $\{C_1\}$ **else** $\{C_2\}$ $\{\psi\}$. Show that any instance of the variant in a proof can be replaced by an instance of the original rule and instances of Implied. Is the converse true as well?

4. A formula φ is *stronger* than a formula φ' (and φ' is *weaker* than φ) if $\varphi \to \varphi'$ is valid over the integers, i.e., every state that satisfies φ also satisfies φ'. So a stronger formula is true in fewer states and a weaker one in more states: $\top$ is the weakest formula and $\bot$ the strongest.

   As in the lectures, a *precondition* is a formula, the first component of a Hoare triple $\{\varphi\}\ P\ \{\psi\}$; the triple may or may not be correct. We call φ a *correct precondition* for a program P and a postcondition ψ if the triple $\{\varphi\}\ P\ \{\psi\}$ holds under partial correctness, i.e., $\models_{par} \{\varphi\}\ P\ \{\psi\}$. The *weakest precondition* for P and ψ is the correct precondition that is weaker than all other correct preconditions (it is unique up to logical equivalence). This problem explains the name and makes this definition precise.

   (a) Order the formulas $x > 10$, $x > 4$, $x = 7$, $\top$ and $\bot$ by strength. Which pairs are incomparable?

   (b) Let P be $x := x + 1$ and ψ be $x > 5$. (i) Show that $x > 4$, $x > 10$, $x = 7$ and $\bot$ are correct preconditions for P and ψ, and that $x > 3$ is not. (ii) Which of these correct preconditions is the weakest? (iii) What precondition does the Assignment rule give for P and ψ? Simplify it and compare it with your answer to (ii).

   Write $[\![\varphi]\!]$ for the set of states that satisfy a formula φ. To talk about sets of states directly, we call a set S of states *safe* for P and ψ if for every state $\alpha \in S$: *if* P started in α terminates, then the final state satisfies ψ. So φ is a correct precondition for P and ψ if and only if $[\![\varphi]\!]$ is safe for P and ψ. Let $\mathrm{wp}(P, \psi)$ be the set of all states α such that, *if* P started in α terminates, then the final state satisfies ψ. A formula ω with $[\![\omega]\!] = \mathrm{wp}(P, \psi)$, if there is one, is the weakest precondition.

   (c) Prove, for every program P and formula ψ: (i) the set $\mathrm{wp}(P, \psi)$ is safe for P and ψ; (ii) every safe set S for P and ψ satisfies $S \subseteq \mathrm{wp}(P, \psi)$. Conclude that a formula φ is a correct precondition for P and ψ if and only if $[\![\varphi]\!] \subseteq \mathrm{wp}(P, \psi)$.

   (d) Which formula is a correct precondition for *every* P and ψ and is the strongest formula of all? Why is it useless? Explain why, because of this, we look for the *weakest* precondition and not for the strongest.

Part 2

5. Recall the rule Partial-while: from $\{\eta \wedge B\}\ C\ \{\eta\}$ conclude $\{\eta\}$ **while** B $\{C\}$ $\{\eta \wedge \neg B\}$.

   (a) Give a formula η that is an invariant of *every* loop, i.e., $\{\eta \wedge B\}\ C\ \{\eta\}$ holds under partial correctness for all B and C. Why does such an invariant usually not help in proving the postcondition we want?

   (b) Prove $\vdash_{par} \{\top\}$ **while** $(\top)$ $\{x := x + 1\}$ $\{x = 0\}$ (Problem 5(c) of Problem Set 8). Which invariant do you use, and why does it work here although it usually does not?

   (c) Prove $\vdash_{par} \{x = 5\}$ **while** $(x \neq 0)$ $\{x := x - 2\}$ $\{\bot\}$ (Problem 5(d) of Problem Set 8). Show that your answer to (a) does not work as an invariant here, and find one that does.

   (d) The derivation in (c) shows that the loop never terminates from a state with $x = 5$. Explain why, using Problem 8(c) of Problem Set 8 and the soundness of the proof calculus. Can a derivation of $\vdash_{par}$ ever show that a program *does* terminate?

6. Consider the programs

$$\textbf{Fac}_2: \quad \begin{array}{l} y := 1; \\ \textbf{while } (x \neq 0) \ \{ \\ \quad y := y * x; \\ \quad x := x - 1 \\ \} \end{array} \qquad\qquad \textbf{Multi}_2: \quad \begin{array}{l} z := 0; \\ \textbf{while } (y \neq 0) \ \{ \\ \quad z := z + x; \\ \quad y := y - 1 \\ \} \end{array}$$

(a) Is $y = x!$ an invariant of the loop in $\textbf{Fac}_2$? Find an invariant that works (hint: use the logical variable x_0) and prove $\vdash_{par} \{x = x_0 \wedge x \geq 0\}\ \textbf{Fac}_2\ \{y = x_0!\}$. Where does your proof use $x \geq 0$?

(b) Prove $\vdash_{par} \{y = y_0 \wedge y \geq 0\}\ \textbf{Multi}_2\ \{z = x \cdot y_0\}$. Why can the postcondition not simply be $z = x \cdot y$?

7. A state α assigns an integer to every variable (also to the logical variables, whose values no program changes). Programs are deterministic: write $\alpha \to_C \alpha'$ if the program C, started in α, terminates in the state α' (there is at most one such α'). The terminating runs of compound programs are given by the following clauses (Lecture 19):

   - $\alpha \to_{C_1;C_2} \alpha''$ if and only if there is a state β with $\alpha \to_{C_1} \beta$ and $\beta \to_{C_2} \alpha''$;
   - $\alpha \to_{\textbf{if } B \ \{C_1\} \ \textbf{else} \ \{C_2\}} \alpha'$ if and only if ($\alpha \models B$ and $\alpha \to_{C_1} \alpha'$) or ($\alpha \models \neg B$ and $\alpha \to_{C_2} \alpha'$);
   - $\alpha \to_{\textbf{while } B \ \{C\}} \alpha'$ if and only if there are $k \geq 0$ and states $\alpha_0 = \alpha, \alpha_1, \ldots, \alpha_k = \alpha'$ such that $\alpha_i \models B$ and $\alpha_i \to_C \alpha_{i+1}$ for all $i < k$, and $\alpha_k \models \neg B$.

   We only consider partial correctness. Recall the notation of Problem 4: $[\![\varphi]\!]$ is the set of states that satisfy φ, and a set S of states is *safe* for C and ψ if $\alpha' \models \psi$ for every $\alpha \in S$ and every α' with $\alpha \to_C \alpha'$. So $\models_{par} \{\varphi\}\ C\ \{\psi\}$ holds if and only if $[\![\varphi]\!]$ is safe for C and ψ. As in Lecture 20, a Boolean expression B is also used as a formula, e.g., in $\varphi \wedge B$. A proof rule is *sound* if for every instance of it, the conclusion holds ($\models_{par}$) whenever all premises hold ($\models_{par}$). (Soundness of the whole calculus, i.e., that $\vdash_{par} \{\varphi\}\ C\ \{\psi\}$ implies $\models_{par} \{\varphi\}\ C\ \{\psi\}$, then follows by induction on derivations. Soundness of Assignment and Implied was Problems 9 and 8(b) of Problem Set 8; for Implied, note that every $\vdash_{AR}$ implication is valid over the integers.) Prove that the following rules are sound, i.e., prove these implications for all formulas φ, η, ψ, all Boolean expressions B and all programs C, C_1, C_2.

   (a) Sequential Composition: if $\models_{par} \{\varphi\}\ C_1\ \{\eta\}$ and $\models_{par} \{\eta\}\ C_2\ \{\psi\}$, then $\models_{par} \{\varphi\}\ C_1; C_2\ \{\psi\}$.

   (b) If-then-Else: if $\models_{par} \{\varphi \wedge B\}\ C_1\ \{\psi\}$ and $\models_{par} \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}$, then $\models_{par} \{\varphi\}\ \textbf{if } B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}$.

   (c) Partial-while: if $\models_{par} \{\eta \wedge B\}\ C\ \{\eta\}$, then $\models_{par} \{\eta\}\ \textbf{while } B\ \{C\}\ \{\eta \wedge \neg B\}$.